AJAY.I - 71812401007 · MODULE 1 – FOUNDATIONS OF AGENTIC AI SYSTEMS NOTES

Syllabus coverage

1. Evolution of AI Agents
2. Symbolic AI to Generative Agents
3. Agentic AI Fundamentals
4. Observe–Orient–Decide–Act (OODA) Loop
5. Perception–Brain–Action Cycle
6. Cognitive Architectures
7. CoALA Framework
8. Working, Episodic and Procedural Memory
9. Action Space
10. Neuro-Symbolic Integration
11. Tool Calling and Structured Outputs
12. Gorilla Framework
13. Python-Based Agent Development

1. Evolution of AI Agents

1.1 What is Artificial Intelligence?

Simple meaning:

Artificial Intelligence is nothing but making computers perform tasks that normally require human intelligence.

For example, humans can understand language, recognize objects, solve problems and make decisions. AI systems are designed to perform these kinds of tasks.

Technical explanation:

Artificial Intelligence (AI) is a branch of computer science that develops systems capable of performing tasks involving learning, reasoning, perception, problem-solving and decision-making.

Examples:

- Recognizing faces in images.
- Understanding spoken language.
- Predicting weather.
- Playing chess.
- Answering questions.

1.2 What is an AI Agent?

Simple meaning:

An AI agent is nothing but a computer system that observes what is happening, thinks about what to do and performs an action to achieve a goal.

Imagine a robot vacuum cleaner.

It observes dirt on the floor, decides to move towards that location and starts cleaning.

It does not simply provide information. It actually performs an action.

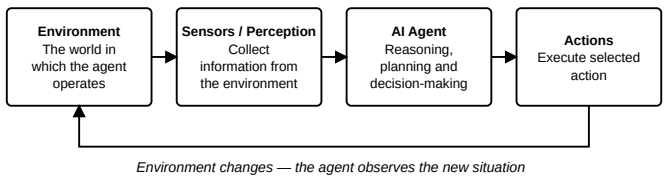
Technical explanation:

An AI agent is an autonomous computational system that perceives its environment through sensors, processes information, selects actions and uses actuators or tools to achieve specified goals.

An agent may operate using rules, search algorithms, machine learning or large language models.

Basic AI agent diagram

DIAGRAM



Basic perception–decision–action interaction

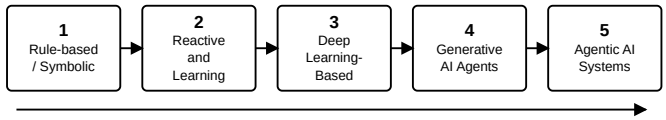
Important components

Component	Simple meaning	Technical meaning
Environment	The world around the agent	External system in which the agent operates
Sensors	Things that collect information	Mechanisms used to perceive the environment
Perception	Understanding collected information	Processing observations into useful representations
Reasoning	Thinking about what to do	Applying logic, inference or learned models
Planning	Choosing steps to reach a goal	Constructing an action sequence
Actuators / Tools	Things used to perform actions	Mechanisms through which the agent affects the environment
Goal	What the agent wants to achieve	Desired target state or objective

1.3 Evolution of AI Agents

AI agents have developed from simple rule-based systems into intelligent systems capable of reasoning, learning, using tools and performing complex tasks.

DIAGRAM



Evolution of AI agents

Classification based on evolution

1. Rule-Based / Symbolic Agents
   Follow predefined rules and logical instructions.
2. Reactive and Learning Agents
   Respond to environmental changes and learn from experience.
3. Deep Learning-Based Agents
   Use neural networks to recognize patterns and process complex information.
4. Generative AI Agents
   Use generative models such as LLMs to understand instructions and generate responses.
5. Agentic AI Systems
   Use reasoning, planning, memory and tools to work towards goals with limited human intervention.

Stage 1: Symbolic AI

Simple meaning:

Symbolic AI is nothing but making computers think using rules written by humans.

For example:

If the temperature is above 38°C, display a warning.

Technical explanation:

Symbolic AI represents knowledge using symbols, logical rules and structured representations. It performs reasoning by applying predefined rules to available information.

Example:

Rule

```
temperature = 39
if temperature > 38:
    print("High temperature")
else:
    print("Normal temperature")
```

Output:

```
High temperature
```

The computer follows the predefined condition.

Advantages:

- Easy to understand.
- Decisions are explainable.
- Suitable for clearly defined problems.

Limitations:

- Cannot easily handle unknown situations.
- Requires humans to define rules.
- Does not automatically learn from experience.

Stage 2: Reactive and Learning Agents

Simple meaning:

These agents respond to changes in the environment. Some can improve their decisions by learning from previous experiences.

Technical explanation:

Reactive agents select actions based on current observations or predefined policies. Learning agents improve their behavior using data, feedback or interaction with the environment.

Example:

A robot detects an obstacle and changes direction.

A reinforcement learning agent may learn which actions produce better rewards.

Stage 3: Deep Learning-Based Agents

Simple meaning:

These agents learn patterns from large amounts of data instead of depending only on manually written rules.

Technical explanation:

Deep learning agents use multilayer neural networks to learn representations and perform tasks such as image recognition, speech processing, prediction and decision-making.

Example:

A self-driving system uses neural networks to recognize vehicles, pedestrians and road signs.

Stage 4: Generative AI Agents

Simple meaning:

Generative AI agents can understand instructions and generate new information, such as text, code or plans.

Technical explanation:

Generative AI agents use models such as Large Language Models (LLMs) to interpret natural-language instructions, generate responses and support reasoning or task execution.

Example:

A user asks:

"Write Python code to calculate student marks."

The model generates the required code.

Stage 5: Agentic AI Systems

Simple meaning:

Agentic AI is nothing but AI that can work towards a goal by deciding what steps to perform and using tools to perform them.

Technical explanation:

Agentic AI systems combine perception, reasoning, planning, memory and action execution to pursue goals in an environment. They may operate autonomously or with human supervision.

Example:

A travel agent system can:

1. Understand a user's destination.
2. Search for available flights.
3. Compare prices.
4. Suggest an itinerary.
5. Book a flight when authorized.

Generative AI vs Agentic AI

Generative AI	Agentic AI
Generates content	Works towards a goal
Usually responds to prompts	Can perform multiple steps
May only provide an answer	Can use tools to execute actions
Does not necessarily maintain task state	Can maintain memory and task progress
Example: Generates travel suggestions	**Example:** Searches and organizes a trip

Important: Generative AI can be a component of an agentic system, but not every generative AI application is an agent.

2. Agentic AI Fundamentals

2.1 What is Agentic AI?

Simple meaning:

Agentic AI is nothing but an AI system that can understand a goal, decide the steps required and perform those steps using available tools.

For example, instead of merely explaining how to calculate an average, an agent can open a dataset, calculate the average and return the result.

Technical explanation:

Agentic AI refers to systems that combine AI reasoning with goal-directed planning, environmental interaction, tool usage, memory and action execution.

These systems can perform multi-step tasks and adapt their actions according to feedback.

2.2 Main characteristics of Agentic AI

1. Autonomy

Simple meaning: The agent can perform tasks without asking the user about every small step.

Technical meaning: Autonomy is the ability to select and execute actions independently within a defined level of control.

2. Goal-oriented behavior

Simple meaning: The agent works towards a specific target.

Technical meaning: The agent uses objectives or desired states to guide its planning and action selection.

3. Perception

Simple meaning: The agent collects information about what is happening.

Technical meaning: Perception converts environmental observations into information that the agent can process.

4. Reasoning

Simple meaning: The agent thinks about the available information before deciding.

Technical meaning: Reasoning involves inference, evaluation and problem-solving to determine suitable actions.

5. Planning

Simple meaning: The agent decides which steps it should perform and in what order.

Technical meaning: Planning constructs an action sequence or policy to achieve a desired goal.

6. Memory

Simple meaning: The agent stores information so it can use it later.

Technical meaning: Memory maintains relevant information about current tasks, previous experiences and learned procedures.

7. Tool usage

Simple meaning: The agent uses external software to perform tasks.

Technical meaning: Tool usage enables an agent to invoke external functions, APIs, databases or software environments to obtain information or execute actions.

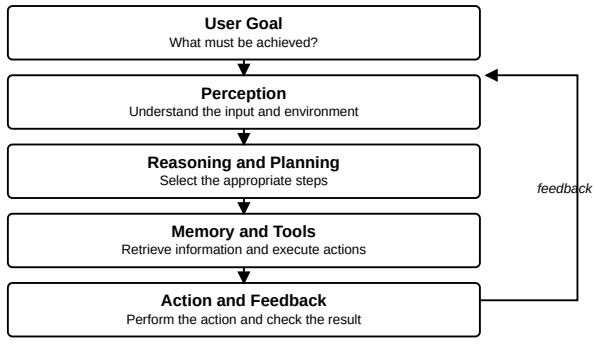
8. Feedback and adaptation

Simple meaning: The agent checks what happened after an action and changes its next decision if needed.

Technical meaning: Feedback provides information about action outcomes, enabling the agent to update its state, revise plans or improve its behavior.

2.3 Basic architecture of an Agentic AI system

DIAGRAM



Basic architecture of an Agentic AI system

Example: AI coding agent

User asks:

"Create a Python program that calculates the sum of two numbers."

The agent:

1. Understands the request.
2. Plans to create a Python program.
3. Generates the code.
4. Uses a Python execution tool.
5. Checks the output.

6. Returns the working code.

This is agentic behavior because the system moves beyond generating an answer and performs a sequence of goal-directed actions.

3. Observe–Orient–Decide–Act (OODA) Loop

3.1 What is OODA?

Simple meaning:

OODA is nothing but a cycle in which an agent observes what is happening, understands the situation, decides what to do and performs the action.

After performing the action, it observes the new situation and repeats the cycle.

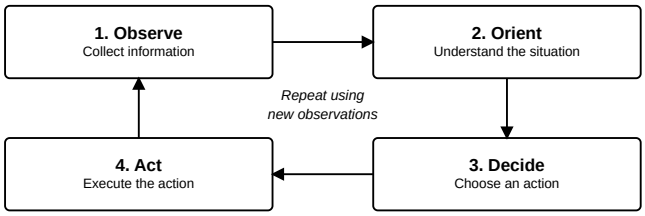
Technical explanation:

The OODA loop is a decision-making framework consisting of four stages: Observe, Orient, Decide and Act.

It enables an intelligent agent to respond to changing environmental conditions through continuous observation and action.

3.2 Four stages of OODA

DIAGRAM



The OODA loop

Stage 1: Observe

Simple meaning: The agent collects information about the environment.

Technical explanation: Observation involves gathering data through sensors, user input, APIs, files or other information sources.

Example:

A robot observes that an obstacle is present in front of it.

Stage 2: Orient

Simple meaning: The agent understands what the collected information means.

Technical explanation: Orientation interprets observations using knowledge, context, memory and environmental models to determine the current situation.

Example:

The robot recognizes that the obstacle blocks its current path.

Stage 3: Decide

Simple meaning: The agent selects the best action.

Technical explanation: Decision-making evaluates available actions and selects one according to the goal, current state and decision policy.

Example:

The robot decides to turn left.

Stage 4: Act

Simple meaning: The agent performs the selected action.

Technical explanation: Action execution uses actuators, tools or software functions to modify the environment.

Example:

The robot turns left and moves forward.

After acting, the robot observes its new position and starts another OODA cycle.

3.3 OODA loop example

Consider an autonomous car.

Stage	Action
Observe	Detect a red traffic light
Orient	Understand that the light requires stopping
Decide	Select the braking action
Act	Apply brakes
Repeat	Observe the next traffic condition

3.4 Advantages of OODA

- Helps agents respond to changing environments.
- Supports continuous decision-making.
- Uses feedback from previous actions.
- Improves situational awareness.
- Helps autonomous systems select appropriate actions.

Exam point: OODA is a continuous loop, not a one-time sequence.

4. Perception–Brain–Action Cycle

4.1 What is the Perception–Brain–Action Cycle?

Simple meaning:

It is nothing but the way an intelligent system receives information, thinks about it and performs an action.

For example, a robot sees a person approaching, understands that the person is nearby and moves backward.

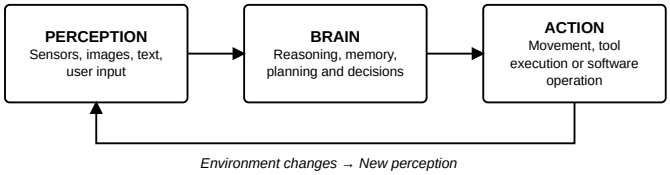
Technical explanation:

The Perception–Brain–Action Cycle is a fundamental agent architecture in which an agent continuously collects environmental information, processes it through a decision-making component and executes actions to achieve its objectives.

The cycle repeats as the environment changes.

4.2 Three main components

DIAGRAM



Perception–Brain–Action Cycle

1. Perception

Simple meaning: Collecting information from the environment.

Technical explanation: Perception processes observations from sensors or input sources and converts them into useful information.

Examples:

- Camera images.
- Microphone audio.
- User instructions.
- Sensor readings.
- Webpage content.

2. Brain

Simple meaning: The brain is the part that thinks and decides.

Technical explanation: The brain component performs reasoning, memory retrieval, planning, decision-making and action selection.

In modern AI agents, this component may use an LLM, a neural network, a rule-based system or a combination of these.

3. Action

Simple meaning: The action component performs what the brain decides.

Technical explanation: Action execution uses actuators, tools, APIs or software functions to change the environment or produce a result.

Examples:

- Moving a robot.
- Sending an email.
- Running Python code.
- Searching a website.
- Updating a database.

4.3 OODA vs Perception–Brain–Action

OODA	Perception–Brain–Action
Observe	Perception
Orient	Brain
Decide	Brain
Act	Action

The Perception–Brain–Action Cycle is a general agent architecture.

OODA provides a more detailed decision-making structure by separating observation, orientation, decision and action.

5. Cognitive Architectures

5.1 What is a Cognitive Architecture?

Simple meaning:

A cognitive architecture is nothing but the design of an AI system that explains how it thinks, remembers, learns and makes decisions.

Just like the human brain has different functions for memory, thinking and decision-making, an AI system can also be designed with different components for these functions.

Technical explanation:

A cognitive architecture is a computational framework that organizes the internal components of an intelligent system, including perception, memory, reasoning, learning, planning and action selection.

It defines how these components interact to produce intelligent behavior.

5.2 Main components of a cognitive architecture

Component	Function
Perception	Receives environmental information
Working memory	Holds information currently being processed
Long-term memory	Stores information for future use
Reasoning	Processes information and draws conclusions
Planning	Determines action sequences
Action selection	Chooses the next action
Learning	Improves behavior through experience

5.3 Types of Cognitive Architectures

1. Symbolic Cognitive Architecture

Simple meaning: Uses rules and logical instructions to think.

Technical explanation: Symbolic architectures represent knowledge through symbols, rules and logical structures.

Example: An expert system that diagnoses faults using predefined rules.

2. Connectionist Architecture

Simple meaning: Uses networks of connected artificial neurons to learn patterns.

Technical explanation: Connectionist architectures use neural networks to represent information and perform learning through adjustments to network parameters.

Example: A neural network recognizing objects in images.

3. Hybrid Cognitive Architecture

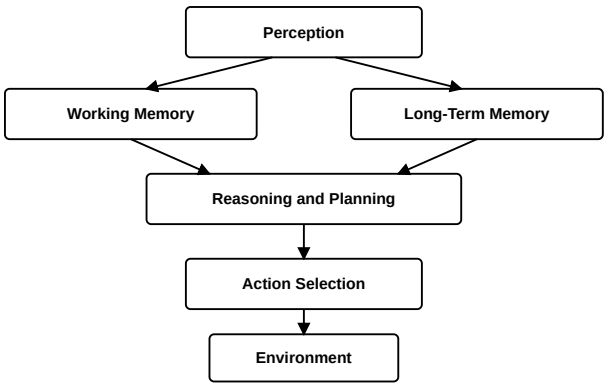
Simple meaning: Combines rule-based thinking with learning from data.

Technical explanation: Hybrid architectures integrate symbolic reasoning with neural or statistical methods to combine structured knowledge and learned representations.

Example: A robot uses neural networks to recognize objects and logical rules to select safe actions.

5.4 Cognitive Architecture Diagram

DIAGRAM



Cognitive architecture

Example

A robot needs to move to a destination.

1. Perception detects the destination.
2. Working memory stores the current location.
3. Long-term memory provides knowledge about navigation.
4. Reasoning determines a suitable route.
5. Planning selects movement steps.
6. Action selection chooses the next movement.
7. The robot executes the movement.

6. CoALA Framework

6.1 What is CoALA?

Simple meaning:

CoALA is nothing but a way of organizing an AI agent so that it can think, remember information, use tools and perform actions.

It helps us understand how an agent's internal memory and decision-making work together.

Technical explanation:

CoALA stands for Cognitive Architectures for Language Agents.

It is a framework for understanding and designing language-based agents by organizing their memory, reasoning, action and learning processes.

CoALA adapts ideas from cognitive architectures to language agents.

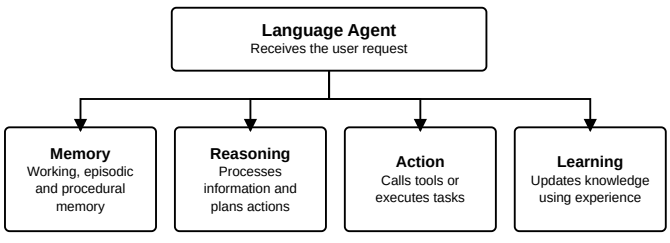
6.2 Main components of CoALA

CoALA describes an agent using four important components:

1. Memory
2. Internal reasoning
3. Action
4. Learning

CoALA architecture

DIAGRAM



CoALA architecture

1. Memory

Simple meaning: Stores information that the agent needs.

Technical explanation: Memory maintains task information, previous experiences and reusable procedures that support reasoning and action.

2. Internal reasoning

Simple meaning: The agent thinks about the problem before performing an action.

Technical explanation: Internal reasoning uses the language model and stored information to interpret instructions, evaluate possibilities and plan actions.

3. Action

Simple meaning: The agent performs a task using a tool or function.

Technical explanation: Action enables the agent to interact with external environments through APIs, software functions and other tools.

4. Learning

Simple meaning: The agent uses previous results to improve its future decisions.

Technical explanation: Learning updates the agent's knowledge, memory or behavior based on feedback and experience.

6.3 CoALA example

User asks:

"Find the average of these numbers: 10, 20, 30."

The agent:

1. Understands the request.
2. Retrieves the required calculation procedure.
3. Plans to calculate the sum and divide by the number of values.
4. Uses a calculator or Python tool.
5. Returns the average.
6. Stores relevant information if future memory is needed.

Important: CoALA is an architectural framework for language agents. It is not simply a chatbot framework or a single Python library.

7. Working, Episodic and Procedural Memory

Memory is an important component of an AI agent because it helps the agent retain information and use it during future decisions.

7.1 What is Memory in AI Agents?

Simple meaning:

Memory is nothing but the information an AI agent keeps so it does not have to start from zero every time.

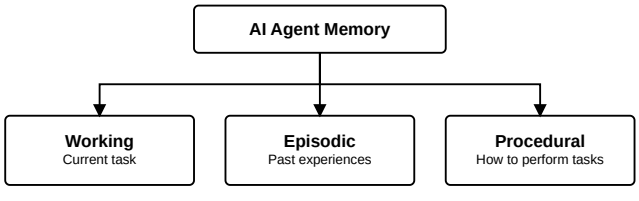
For example, if an agent remembers that a user prefers Python, it can use Python in future coding tasks.

Technical explanation:

Memory in AI agents is a mechanism for storing, retrieving and updating information used for reasoning, planning, learning and task execution.

7.2 Types of Memory

DIAGRAM



Types of AI agent memory

1. Working Memory

Simple meaning:

Working memory is nothing but the information the agent is currently using to complete a task.

Imagine solving a mathematics problem. You temporarily remember the numbers and calculations while solving it.

Technical explanation:

Working memory stores information that is currently relevant to the agent's active task.

It maintains temporary task context, intermediate reasoning results and information required for immediate decision-making.

Example:

User asks:

"Calculate the average of 10, 20 and 30."

The agent temporarily stores:

- Numbers: 10, 20, 30.
- Sum: 60.
- Count: 3.
- Average: 20.

This information is used while solving the current problem.

2. Episodic Memory

Simple meaning:

Episodic memory is nothing but remembering what happened in the past.

For example, an agent remembers that it previously searched for a particular book.

Technical explanation:

Episodic memory stores information about specific events, interactions, experiences and their outcomes.

It helps the agent use past experiences when making future decisions.

Example:

An AI assistant remembers:

"Yesterday, the user asked for Python code to analyze a CSV file."

The agent can use this previous interaction as context when the user asks a related question.

3. Procedural Memory

Simple meaning:

Procedural memory is nothing but remembering how to perform a task.

For example, remembering the steps required to install Python.

Technical explanation:

Procedural memory stores reusable procedures, skills, action sequences and task execution instructions.

It helps the agent perform repeated tasks efficiently.

Example:

An agent remembers the procedure for analyzing a CSV file:

1. Load the file.
2. Check the columns.
3. Clean missing values.
4. Perform analysis.
5. Display results.

7.3 Comparison

Memory type	Stores	Example
Working memory	Current task information	Numbers being calculated
Episodic memory	Past experiences	Previous user interaction
Procedural memory	How to perform a task	Steps for analyzing a dataset

Exam point: Working memory is temporary task context, episodic memory stores experiences, and procedural memory stores reusable skills or procedures.

8. Action Space

8.1 What is Action Space?

Simple meaning:

Action space is nothing but the list of actions an AI agent is allowed to perform.

For example, a robot may move forward, move backward, turn left or turn right.

These possible movements form its action space.

Technical explanation:

Action space is the set of all possible actions available to an agent in a given environment.

The agent selects an action from this set based on its current state, goal and decision policy.

8.2 Types of Action Space

1. Discrete Action Space

Simple meaning: The agent has a fixed number of possible actions.

Technical explanation: A discrete action space contains a finite or countable set of distinct actions.

Example: A robot can choose:

- Move forward.
- Move backward.
- Turn left.
- Turn right.

2. Continuous Action Space

Simple meaning: The agent can select actions using values within a range.

Technical explanation: A continuous action space allows actions represented by continuous numerical values.

Example:

A robot selects a movement speed between 0 and 100.

3. Hybrid Action Space

Simple meaning: The agent has both fixed actions and numerical actions.

Technical explanation: A hybrid action space combines discrete decisions with continuous control values.

Example:

A robot selects a movement direction and a speed.

8.3 Action Space Diagram

Agent — Selects an action → **Forward / Backward / Turn** → **Environment** — The selected action changes the environment

8.4 Example

A game-playing agent has the following action space:

```
actions = ["left", "right", "jump", "stay"]
action = "jump"
print("Selected action:", action)
```

Output:

```
Selected action: jump
```

The agent selects one available action from its action space.

9. Neuro-Symbolic Integration

9.1 What is Neuro-Symbolic AI?

Simple meaning:

Neuro-Symbolic AI is nothing but combining the learning ability of neural networks with the logical thinking ability of rule-based systems.

Neural networks recognize patterns, while symbolic systems apply rules.

Combining them helps an AI system perform both tasks.

Technical explanation:

Neuro-Symbolic AI integrates neural learning methods with symbolic knowledge representation and logical reasoning.

Neural components learn patterns from data, while symbolic components represent explicit facts, rules and relationships.

9.2 Neural AI

Simple meaning: Learns patterns from examples.

Technical explanation: Neural AI uses artificial neural networks to learn representations and relationships from training data.

Example:

A neural network recognizes whether an image contains a cat.

9.3 Symbolic AI

Simple meaning: Uses rules and facts to make decisions.

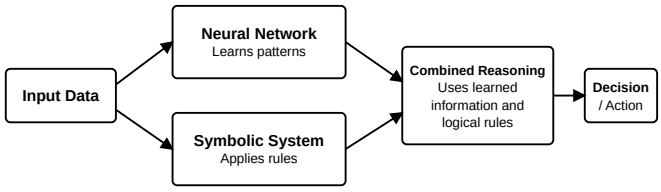
Technical explanation: Symbolic AI represents knowledge through symbols, logic and predefined rules.

Example:

```
if temperature > 38:
    print("High temperature")
```

9.4 Neuro-Symbolic Architecture

DIAGRAM



Neuro-Symbolic architecture

9.5 Example

Consider a smart manufacturing system.

The neural network detects that a machine is producing unusual vibration patterns.

The symbolic system applies a rule:

"If vibration exceeds the safe threshold, stop the machine."

The combined system can detect the pattern and apply the safety rule.

9.6 Advantages

- Combines learning and reasoning.
- Can use structured knowledge.
- Supports explainable decision-making.
- Helps solve complex problems.
- Can improve reliability when rules are properly designed.

10. Tool Calling and Structured Outputs

10.1 What is Tool Calling?

Simple meaning:

Tool calling is nothing but an AI agent using another program to perform a task.

For example, if an agent needs to calculate a large number, it can call a calculator instead of trying to calculate everything internally.

Technical explanation:

Tool calling is a mechanism through which an AI model selects and invokes an external function, API or software tool to obtain information or execute an action.

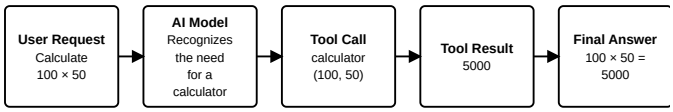
The tool returns a result, which the agent can use for further reasoning.

Examples of tools

- Calculator.
- Python interpreter.
- Web search.
- Database.
- File reader.
- Email API.
- Weather API.

10.2 How Tool Calling Works

DIAGRAM



The tool result is returned to the model for the final answer

How tool calling works

Tool calling steps

1. User provides a request.
2. The AI model understands the request.
3. The model selects an appropriate tool.
4. The system executes the tool.
5. The tool returns a result.
6. The AI model uses the result.
7. The agent produces the final answer.

10.3 Python Tool Calling Example

This is a simple example of an agent using a Python function as a tool.

```
def calculator(a, b):
    return a + b

result = calculator(10, 20)
print(result)
```

Output:

```
30
```

The calculator function acts as the tool.

The agent can call it when a calculation is required.

Important: This example demonstrates a tool function. A complete LLM-based agent additionally needs a model that decides when and how to call the function.

10.4 What are Structured Outputs?

Simple meaning:

Structured output means making the AI return information in a fixed format instead of writing it in any form it wants.

For example, instead of answering:

"Ajay scored 90 marks and passed."

The AI returns:

```
{
  "name": "Ajay",
  "marks": 90,
  "result": "Pass"
}
```

Technical explanation:

Structured outputs constrain an AI model's response to a predefined schema, such as JSON, ensuring that the generated information follows the required structure and data types.

This makes the output easier for software applications to process.

Example: Student result

```
import json

result = {
    "name": "Ajay",
    "marks": 90,
    "result": "Pass"
}
print(json.dumps(result, indent=2))
```

Output:

```
{
  "name": "Ajay",
  "marks": 90,
  "result": "Pass"
}
```

Tool Calling vs Structured Output

Tool Calling	Structured Output
Executes an external function	Produces information in a fixed format
Used to perform actions	Used to organize information
Example: Calculator	**Example:** JSON response
Returns tool results	Returns structured data

Exam point: Tool calling enables action execution, while structured outputs enable reliable machine-readable responses.

11. Gorilla Framework

11.1 What is Gorilla?

Simple meaning:

Gorilla is nothing but a framework and research project that helps AI models learn how to use external tools and APIs.

For example, an AI model may know how to answer questions, but it may not know how to call a weather API correctly.

Gorilla helps improve the model's ability to select and use APIs.

Technical explanation:

Gorilla is an open-source research project focused on improving Large Language Models' ability to use external APIs through API retrieval, function selection and accurate function-call generation.

It helps language models interact with external tools by learning from API documentation and function descriptions.

11.2 Why is Gorilla needed?

An ordinary language model can generate text, but external tools require precise function names and parameters.

For example, a weather API may require:

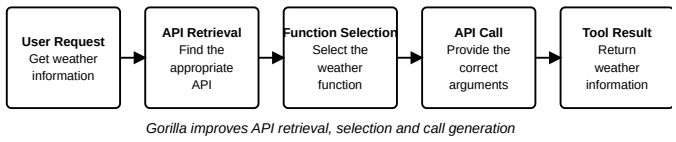
- Location.
- Date.
- Temperature unit.

If the model provides incorrect parameter names, the tool call may fail.

Gorilla addresses this challenge by improving API selection and function-call generation.

11.3 Working of Gorilla

DIAGRAM



Gorilla improves API retrieval, selection and call generation

Working of Gorilla

Steps

1. The user provides a request.
2. The model identifies the required external functionality.
3. The system retrieves relevant API information.
4. The model selects the appropriate API.
5. The model generates the function call.
6. The external API executes the request.
7. The result is returned to the agent.

11.4 Main features

API retrieval

Gorilla can use API documentation to identify the relevant function for a task.

Function selection

It helps select an appropriate API from available choices.

Function-call generation

It generates the required function name and arguments.

Improved API usage

It aims to reduce incorrect API calls and improve the accuracy of tool interaction.

11.5 Example

User asks:

"Get the weather for Coimbatore."

The agent needs a weather API.

A suitable function call might look like:

```
get_weather(
    location="Coimbatore",
    unit="Celsius"
)
```

The weather service returns the result.

The agent then uses that result to answer the user.

Important: Gorilla is not itself a weather service. It helps language models select and use APIs.

12. Python-Based Agent Development

This is an important practical topic because it connects the concepts of perception, reasoning, decision-making and action.

12.1 What is Python-Based Agent Development?

Simple meaning:

Python-based agent development means using Python to create a computer program that observes information, makes decisions and performs actions.

Technical explanation:

Python-based agent development involves implementing agent components such as perception, state representation, decision-making, memory, tool execution and feedback using Python libraries and functions.

12.2 Basic Agent Structure

A simple agent can be represented as:

Input → Perception → Decision-Making → Action → Output

12.3 Simple Python Agent Using OODA

This program demonstrates an agent that observes a temperature, understands the condition, decides what to do and performs an action.

```
# Simple AI Agent using OODA

# Observe
temperature = 39

# Orient
if temperature > 38:
    situation = "High temperature"
else:
    situation = "Normal temperature"

# Decide
if situation == "High temperature":
    action = "Turn on cooling"
else:
    action = "Keep cooling off"

# Act
print("Observation:", temperature)
print("Situation:", situation)
print("Action:", action)
```

Output

```
Observation: 39
Situation: High temperature
Action: Turn on cooling
```

Explanation

Observe: The agent receives the temperature value.

Orient: It determines whether the temperature is high or normal.

Decide: It selects an appropriate action.

Act: It prints the selected action.

This is a simple rule-based agent. It demonstrates the OODA cycle but does not use an LLM or learn from experience.

12.4 Python Agent with a Tool

Now consider an agent that can calculate the average of student marks using a Python function.

```
# Tool
def calculate_average(marks):
    return sum(marks) / len(marks)

# Agent
marks = [80, 90, 70]

# Tool calling
average = calculate_average(marks)

# Output
print("Average:", average)
```

Output:

```
Average: 80.0
```

Working

1. The agent receives the marks.
2. It identifies that average calculation is required.
3. It calls the calculation tool.
4. The tool calculates the average.
5. The agent returns the result.

This demonstrates how an agent can use a tool to perform a task.

13. Important Classifications for Module 1

These classifications are useful for short-answer questions, concept comparisons and exam revision.

AI Agent Types

Type	Main working
Simple reflex agent	Uses current observations and rules
Model-based agent	Maintains an internal representation of the environment
Goal-based agent	Selects actions to achieve goals
Utility-based agent	Selects actions based on utility or preference
Learning agent	Improves behavior through experience

Agent Memory

Type	Purpose
Working memory	Stores current task information
Episodic memory	Stores past experiences
Procedural memory	Stores reusable procedures

Action Space

Type	Example
Discrete	Left, right, jump
Continuous	Speed between 0 and 100
Hybrid	Select direction and speed

Cognitive Architecture

Type	Main approach
Symbolic	Rules and logic
Connectionist	Neural networks
Hybrid	Neural learning + symbolic reasoning

Agent Decision Frameworks

Framework	Main purpose
OODA	Observe, understand, decide and act
Perception–Brain–Action	General agent interaction cycle
CoALA	Organizing language-agent memory, reasoning, action and learning

14. Important Differences for Exam Questions

OODA vs CoALA

OODA	CoALA
Decision-making framework	Language-agent architecture framework
Observe, Orient, Decide, Act	Memory, reasoning, action and learning
Focuses on the decision cycle	Focuses on organizing agent components
Useful for dynamic environments	Useful for designing language agents

Working vs Episodic vs Procedural Memory

Working	Episodic	Procedural
Current information	Past events	Task procedures
Temporary task context	Experience-based memory	Skill-based memory
Current calculation	Previous interaction	How to perform a task

Tool Calling vs Gorilla

Tool Calling	Gorilla
General mechanism for using external tools	Research framework for improving API use
Executes functions or APIs	Helps select and generate API calls
Used by many agent systems	Focuses on accurate API interaction